

Assignment 4 Answer

1. In the traffic-light model, if Light1 is removed, does live2 still fail without weak fairness? Explain why or why not.

No. Because Light2 is the only active process left, it is the only one enabled. Therefore, it will definitely be taken and cannot be starved, even without weak fairness.

2. In the binary semaphore model, replace atomic { sem == true -> sem = false } with sem == true -> sem = false. Does mutual exclusion still hold? Explain why or why not.

No, mutual exclusion does not hold. Removing the atomic block breaks the semaphore entirely, allowing both processes to enter the critical section at the same time as both think semaphore is available.

Counterexample:

1. Initial State: sem = true. Both process P and process Q are waiting to acquire the semaphore.
2. Step 2 (Process P): Evaluates sem == true. It evaluates to true. Process P is now cleared to execute the next line, but the scheduler switches away before it updates the variable.
3. Step 3 (Process Q): Evaluates sem == true. Since P hasn't changed it yet, it still evaluates to true. Process Q is also cleared to proceed.
4. Step 4 (Process P): Executes sem = false and enters its critical section (P@critP).
5. Step 5 (Process Q): Executes sem = false (redundantly) and enters its critical section (Q@critQ).
6. Result: P@critP && Q@critQ is true. Mutual exclusion is violated.

3. In Peterson's algorithm, remove turn = Q_TURN from P . Does mutual exclusion still hold? Explain why or why not.

No, mutual exclusion does not hold. Removing this line violates safety, meaning both processes can crash into the critical section at the exact same time. Without turn = Q_TURN, P will not set the turn for Q to enter when it raises its hand signaling it want to enter critical section.

Counterexample:

1. Initial State: Nobody is in the critical section. (wantP = false, wantQ = false).
2. Q wants to enter: Q sets wantQ = true.
3. Q is polite: Q sets turn = P_TURN (giving the turn to P).
4. Q enters: Q checks if it can enter. Since P has not raised its hand yet (wantP is false), Q easily enters its critical section.
5. P wants to enter now: P sets wantP = true. (But P skips setting the turn variable because the line was removed)

6. P checks its lock condition: P checks if it's allowed to enter by looking at its rule: (wantQ == false || turn == P_TURN).

- wantQ is true (because Q is inside).
- turn == P_TURN is also true (because Q politely set it to P_TURN in Step 3, and P never changed it)

7. P enters anyway: Because turn == P_TURN is true, P's lock opens wide. P steps right into its critical section.

8. Result: Both Process P and Process Q are in their critical sections at the same time. Mutual exclusion is broken.